

DOCUMENTACIÓN TÉCNICA DE OPERACIONES

EJB MANAGER

Guía de despliegue a servidor de producción

React 19 + Vite

Express + TypeScript

Prisma ORM

PostgreSQL 16

LiveKit (Huddles)

pnpm workspaces

Proyecto: Innovation Hub · EJB Manager
Documento generado: 9 de septiembre de 2026
Preparado para: santiago241200@gmail.com

0. Contenido

1 Resumen de la arquitectura

2 Requisitos del servidor

3 Preparación del servidor

4 Obtener el código y dependencias

5 Base de datos PostgreSQL

6 Variables de entorno de producción

7 Compilar backend y frontend

8 Ejecutar el backend con PM2

9 Servir el frontend con Nginx + HTTPS

10 LiveKit en producción (Huddles)

11 Firewall y puertos

12 Respaldo de la base de datos

13 Verificación final y checklist

14 Actualizaciones futuras

1. Resumen de la arquitectura

EJB Manager es una aplicación de cuatro capas. El frontend (React/Vite) se compila a archivos estáticos y se sirve por Nginx; el backend (Express) expone la API REST bajo `/api`; Prisma traduce esas operaciones a PostgreSQL; y LiveKit soporta las videollamadas (Huddles).

```
Navegador
  | HTTPS
  ▼
Nginx  — sirve /frontend/dist (estático)
  | proxy_pass /api →
  ▼
Backend Express (Node, puerto 4000, PM2)
  | Prisma Client
  ▼
PostgreSQL 16 (puerto 5432)

Huddles (audio/video): LiveKit Cloud o LiveKit self-hosted (puertos 7880-7882)
```

Importante: el backend NO sirve el HTML del frontend — son dos procesos independientes. En producción, Nginx es responsable de servir los archivos compilados de `frontend/dist` y de redirigir `/api` hacia el backend.

2. Requisitos del servidor

Componente	Requisito mínimo recomendado
Sistema operativo	Ubuntu Server 22.04 LTS o superior (o Windows Server si ya se administra así)
CPU / RAM	2 vCPU / 4 GB RAM (8 GB si se activan Huddles con LiveKit self-hosted)
Disco	40 GB SSD (más espacio si se almacenan muchos documentos adjuntos)
Node.js	20.x LTS o superior
pnpm	9.x
PostgreSQL	16.x
Nginx	1.18 o superior
PM2	Última versión (gestor de procesos Node)
Dominio propio	Necesario para HTTPS con Let's Encrypt (ej. ejbmanager.miempresa.com)

3. Preparación del servidor

Ejemplo sobre Ubuntu 22.04. Ajustar comandos si el servidor usa otra distribución.

3.1 Actualizar el sistema e instalar herramientas base

```
sudo apt update && sudo apt upgrade -y  
sudo apt install -y curl git build-essential
```

3.2 Instalar Node.js 20 LTS

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -  
sudo apt install -y nodejs  
node -v    # debe mostrar v20.x
```

3.3 Instalar pnpm y PM2

```
sudo npm install -g pnpm@9 pm2
```

3.4 Instalar PostgreSQL 16

```
sudo apt install -y postgresql postgresql-contrib  
sudo systemctl enable postgresql --now
```

3.5 Instalar Nginx y Certbot (HTTPS)

```
sudo apt install -y nginx certbot python3-certbot-nginx
```

4. Obtener el código y dependencias

- 1 Clonar (o copiar) el repositorio en el servidor, por ejemplo en `/var/www/ejb-manager`:

```
sudo mkdir -p /var/www/ejb-manager
sudo chown $USER:$USER /var/www/ejb-manager
git clone <URL_DEL_REPOSITORIO> /var/www/ejb-manager
cd /var/www/ejb-manager
```

- 2 Instalar dependencias del monorepo (pnpm workspaces resuelve backend y frontend):

```
pnpm install
```

Si el código no está en un repositorio Git accesible desde el servidor, se puede comprimir el proyecto local (excluyendo `node_modules`, `dist`, `.git`) y subirlo por SCP/SFTP.

5. Base de datos PostgreSQL

5.1 Crear usuario y base de datos de producción

```
sudo -u postgres psql
```

```
CREATE DATABASE ejb_innovation_hub;  
CREATE USER ejb_prod WITH ENCRYPTED PASSWORD 'CAMBIA-ESTA-CONTRASENA';  
GRANT ALL PRIVILEGES ON DATABASE ejb_innovation_hub TO ejb_prod;  
\q
```

Seguridad: usar una contraseña fuerte y distinta a la de desarrollo (`ejb_local` es solo para el `docker-compose.yml` local). No exponer el puerto 5432 a Internet.

5.2 Aplicar el esquema con Prisma

Desde `backend/`, con `DATABASE_URL` ya apuntando a producción (ver sección 6):

```
cd backend  
pnpm prisma:generate  
pnpm exec prisma migrate deploy
```

Se usa `prisma migrate deploy` (no `migrate dev`) en producción: aplica las migraciones existentes sin generar nuevas ni pedir confirmación interactiva.

6. Variables de entorno de producción

Crear `backend/.env` en el servidor (nunca subir este archivo a Git) a partir de `backend/.env.example` :

```
DATABASE_URL="postgresql://ejb_prod:CONTRASENA@localhost:5432/ejb_innovation_hub?
schema=public"
PORT=4000
FRONTEND_URL="https://ejbmanager.miempresa.com"
JWT_SECRET="generar con: openssl rand -base64 48"
LIVEKIT_URL="wss://tu-proyecto.livekit.cloud"
LIVEKIT_API_KEY="clave de LiveKit Cloud"
LIVEKIT_API_SECRET="secreto de LiveKit Cloud"
ENABLE_HUDDLES=true
```

Variable	Uso
<code>DATABASE_URL</code>	Cadena de conexión a PostgreSQL de producción
<code>PORT</code>	Puerto interno del backend (detrás de Nginx)
<code>FRONTEND_URL</code>	Dominio público exacto del frontend; se valida contra CORS
<code>JWT_SECRET</code>	Firma de sesiones; debe tener 32+ caracteres (validado por <code>production-check.mjs</code>)
<code>LIVEKIT_*</code>	Credenciales de LiveKit para Huddles (ver sección 10)

Nota sobre CORS: el backend solo acepta peticiones desde IPs de red local o desde el valor exacto de `FRONTEND_URL` . Si el dominio de producción no coincide exactamente (incluyendo `https://`), la API rechazará las peticiones del navegador.

Validar que la configuración mínima esté completa:

```
cd backend
pnpm production:check
```

7. Compilar backend y frontend

- 1 Compilar ambos paquetes desde la raíz del proyecto:

```
pnpm build
```

Esto ejecuta `tsc` en el backend (genera `backend/dist`) y `vite build` en el frontend (genera `frontend/dist`).

- 2 Verificar que existan las carpetas compiladas:

```
ls backend/dist/server.js  
ls frontend/dist/index.html
```

8. Ejecutar el backend con PM2

PM2 mantiene el backend corriendo en segundo plano, lo reinicia si falla y lo levanta automáticamente al reiniciar el servidor.

- 1 Iniciar el proceso:

```
cd /var/www/ejb-manager/backend  
pm2 start dist/server.js --name.ejb-manager-api
```

- 2 Guardar la lista de procesos y habilitar arranque automático:

```
pm2 save  
pm2 startup # ejecutar el comando que PM2 imprime en pantalla
```

- 3 Comandos útiles de operación diaria:

```
pm2 status # ver estado  
pm2 logs.ejb-manager-api # ver logs en vivo  
pm2 restart.ejb-manager-api # reiniciar tras un cambio
```

9. Servir el frontend con Nginx + HTTPS

9.1 Archivo de configuración

Crear `/etc/nginx/sites-available/ejb-manager`:

```
server {
    listen 80;
    server_name ejbmanager.miempresa.com;

    root /var/www/ejb-manager/frontend/dist;
    index index.html;

    location / {
        try_files $uri /index.html;
    }

    location /api/ {
        proxy_pass http://127.0.0.1:4000/api/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    client_max_body_size 5m;
}
```

9.2 Activar el sitio

```
sudo ln -s /etc/nginx/sites-available/ejb-manager /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx
```

9.3 Habilitar HTTPS con Let's Encrypt

```
sudo certbot --nginx -d ejbmanager.miempresa.com
```

Certbot edita automáticamente el bloque `server` para escuchar en 443 con certificado válido y renovación automática (a diferencia del certificado autofirmado usado en desarrollo local).

10. LiveKit en producción (Huddles)

En desarrollo local, LiveKit corre vía `docker compose up -d livekit` (modo `--dev`, sin TLS). Para producción hay dos rutas:

Opción A — LiveKit Cloud (recomendada, sin infraestructura propia)

1. Crear un proyecto en `cloud.livekit.io`.
2. Copiar la URL WebSocket (`wss:// ...`), API Key y API Secret.
3. Colocarlos en `backend/.env` como `LIVEKIT_URL`, `LIVEKIT_API_KEY`, `LIVEKIT_API_SECRET`.
4. No requiere cambios de código ni puertos adicionales en el servidor.

Opción B — LiveKit self-hosted

Requiere TLS propio y abrir los puertos 7880 (WebSocket/HTTPS), 7881 (TCP) y 7882/UDP (RTC) en el firewall. Se recomienda usar el `docker-compose.yml` del proyecto como base, reemplazando `--dev` por una configuración con certificados válidos según la guía oficial de LiveKit.

Si Huddles no es una prioridad inmediata, se puede desplegar con `ENABLE_HUDDLES=false` y activarlo después sin afectar el resto de la aplicación.

11. Firewall y puertos

Puerto	Servicio	Exposición
80 / 443	Nginx (HTTP/HTTPS)	Pública
4000	Backend Express	Solo <code>localhost</code> (vía proxy de Nginx)
5432	PostgreSQL	Solo <code>localhost</code>
7880-7882	LiveKit (solo si self-hosted)	Pública, con TLS

```
sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full'
sudo ufw enable
```

12. Respaldo de la base de datos

El proyecto incluye un script de respaldo listo para usar:

```
cd backend
pnpm backup # genera backend/backups/ejb-<fecha>.sql usando pg_dump
```

Para automatizarlo diariamente con **cron**:

```
crontab -e
```

```
# respaldo diario a las 2:00 a.m.
0 2 * * * cd /var/www/ejb-manager/backend && /usr/bin/pnpm backup >> /var/log/ejb-
backup.log 2>&1
```

Copiar periódicamente los archivos **.sql** generados fuera del servidor (almacenamiento externo, S3, etc.) para protegerse ante una falla de disco.

13. Verificación final y checklist

Verificación	Comando / URL
Backend responde	<code>curl http://127.0.0.1:4000/api/health</code> → <code>{"status":"ok"}</code>
Frontend carga por HTTPS	<code>https://ejbmanager.miempresa.com</code>
Login funciona end-to-end	Probar con un usuario real desde el navegador
Variables mínimas validadas	<code>pnpm production:check</code> dentro de <code>backend/</code>
PM2 sobrevive un reinicio	<code>sudo reboot</code> y luego <code>pm2 status</code>
Certificado HTTPS válido	Candado verde en el navegador, sin advertencias
Respaldo probado	<code>pnpm backup</code> genera archivo <code>.sql</code> correctamente

14. Actualizaciones futuras

Flujo recomendado para desplegar una nueva versión del código:

- 1 `git pull` en `/var/www/ejb-manager`
- 2 `pnpm install` (por si hay dependencias nuevas)
- 3 `pnpm --dir backend exec prisma migrate deploy` (si hay migraciones nuevas)
- 4 `pnpm build` (recompila backend y frontend)
- 5 `pm2 restart ejb-manager-api` (Nginx sirve el nuevo `frontend/dist` automáticamente, sin reinicio)